

Computer Vision Lab 05 - Code Cheat Sheet

How to use this sheet in the coding exam

For every code problem, identify these four things first:

- **INPUT:** image/data and required format (grayscale, BGR, HSV, pixel array, etc.)
- **OPERATION:** the OpenCV/NumPy function that actually performs the CV operation
- **OUTPUT:** binary mask, edge map, segmented image, markers, histogram, etc.
- **PARAMETERS:** the values most likely to be changed by the question

Lab 5 topic map

Uneven illumination	-> Adaptive Thresholding
Simple intensity split	-> Global Thresholding
Automatic threshold	-> Otsu
Specific color	-> HSV + inRange
Object boundaries	-> Canny
Known starting pixel	-> Region Growing
Touching objects	-> Watershed
Similar pixel features	-> K-Means

Important: The manual has no standalone Python code block for deep-learning segmentation or active contours; do not invent a "manual code" if asked to reproduce code from the manual.

PART A - CODE FROM LAB 5 MANUAL

1. Global Thresholding

Purpose: Convert a grayscale image into a binary image using one threshold for the whole image.

```
import cv2

# Read image as grayscale
```

```
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)

# Apply global binary thresholding
_, binary_mask = cv2.threshold(image, 128, 255, cv2.THRESH_BINARY)
```

Line meanings

- `cv2.imread(..., cv2.IMREAD_GRAYSCALE)` -> loads the image as one intensity channel.
- `cv2.threshold(...)` -> compares every pixel with the same threshold.
- `128` -> manually selected threshold.
- `255` -> output value for pixels that satisfy the binary condition.
- `cv2.THRESH_BINARY` -> binary threshold mode.
- `_` -> first returned value is ignored.
- `binary_mask` -> resulting thresholded image.

Function / argument dictionary

`cv2.imread(filename, flags)`

- `filename` = image path/name.
- `flags` = how to read image; manual uses `cv2.IMREAD_GRAYSCALE`.

`cv2.threshold(src, thresh, maxval, type)`

- `src` = input grayscale image.
- `thresh` = threshold value.
- `maxval` = output value used by the chosen threshold type.
- `type` = thresholding mode, e.g. `cv2.THRESH_BINARY`.
- returns `(threshold_used, output_image)`.

Exam modification

Change only the threshold value when the question asks to compare thresholds:

```
_, result = cv2.threshold(image, 80, 255, cv2.THRESH_BINARY)
```

2. Adaptive Thresholding

Purpose: Use a local threshold instead of one threshold for the entire image; useful when illumination varies.

```
import cv2

# Read image as grayscale
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)

# Apply adaptive thresholding
binary_mask = cv2.adaptiveThreshold(
    image,
    255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11,
    2
)
```

Line meanings

- grayscale input because the threshold is based on intensity.
- `ADAPTIVE_THRESH_GAUSSIAN_C` -> local threshold uses a Gaussian-weighted neighborhood.
- `THRESH_BINARY` -> binary output.
- `11` -> `blockSize`, the local neighborhood size.
- `2` -> `C`, the offset/subtraction term in the adaptive calculation.

Function / argument dictionary

```
cv2.adaptiveThreshold(src, maxValue, adaptiveMethod, thresholdType, blockSize, C)
```

- `src` = input grayscale image.
- `maxValue` = output value used for foreground pixels.

- `adaptiveMethod` = `cv2.ADAPTIVE_THRESH_MEAN_C` or `cv2.ADAPTIVE_THRESH_GAUSSIAN_C` .
- `thresholdType` = commonly `cv2.THRESH_BINARY` or `cv2.THRESH_BINARY_INV` .
- `blockSize` = odd neighborhood size used around each pixel.
- `C` = constant subtracted from the local statistic.

Exam changes

- Mean: `cv2.ADAPTIVE_THRESH_MEAN_C`
- Gaussian: `cv2.ADAPTIVE_THRESH_GAUSSIAN_C`
- Test different odd `blockSize` values, e.g. 7, 11, 15 .
- Test different `C` values, e.g. 2, 5, 8 .

3. Otsu's Thresholding

Purpose: Automatically choose a global threshold from the image histogram.

```
import cv2

# Read image as grayscale
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)

# Apply Otsu's automatic thresholding
threshold, binary_mask = cv2.threshold(
    image,
    0,
    255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

print('Otsu threshold:', threshold)
```

Line meanings

- `0` is supplied because Otsu chooses the threshold automatically.
- `cv2.THRESH_OTSU` activates Otsu's method.

- `threshold` receives the threshold selected by Otsu.
- `binary_mask` is the final binary segmentation.

Function / argument dictionary

```
cv2.threshold(src, thresh, maxval, type)
```

- same four main arguments as global thresholding.
- with `cv2.THRESH_OTSU`, the supplied threshold is ignored/automatically determined.

Histogram pattern

```
import matplotlib.pyplot as plt

plt.hist(image.ravel(), bins=256, range=[0, 256])
plt.xlabel('Intensity')
plt.ylabel('Number of Pixels')
plt.show()
```

`image.ravel()` -> flattens the 2D image into 1D pixel values.

4. Color-Based Thresholding

Purpose: Select pixels lying inside a specified HSV color range.

```
import cv2
import numpy as np

# Read the color image
image = cv2.imread('image.jpg')

# Convert BGR image to HSV
hsv_image = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

# Define lower and upper HSV limits
lower_bound = np.array([30, 50, 50])
upper_bound = np.array([60, 255, 255])
```

```
# Create a binary mask for pixels inside the color range
mask = cv2.inRange(hsv_image, lower_bound, upper_bound)
```

Line meanings

- OpenCV reads normal color images in BGR order.
- `cv2.cvtColor(..., cv2.COLOR_BGR2HSV)` changes representation to HSV.
- `lower_bound` and `upper_bound` define the accepted HSV range.
- `cv2.inRange()` keeps pixels inside that range and rejects pixels outside it.

Optional object extraction

```
result = cv2.bitwise_and(image, image, mask=mask)
```

Function / argument dictionary

`cv2.cvtColor(src, code)`

- `src` = input image.
- `code` = color conversion code; manual uses `cv2.COLOR_BGR2HSV`.

`np.array(object)`

- `object` = list/values used to create the lower/upper bound array.

`cv2.inRange(src, lowerb, upperb)`

- `src` = HSV image.
- `lowerb` = lower bound for every channel.
- `upperb` = upper bound for every channel.
- returns a binary mask.

`cv2.bitwise_and(src1, src2, mask=mask)`

- keeps pixels where the mask is non-zero.

5. Hysteresis Thresholding / Canny Edge Detector

Purpose: Find edges using a low and a high threshold.

```
import cv2

# Read image as grayscale
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)

# Detect edges using Canny
edges = cv2.Canny(image, 100, 200)
```

Line meanings

- grayscale is used because Canny operates on image intensity.
- 100 = lower threshold.
- 200 = upper threshold.
- output is an edge map.

Hysteresis idea

> high threshold	-> strong edge
low to high	-> weak edge; kept when connected appropriately
< low threshold	-> rejected

Function / argument dictionary

```
cv2.Canny(image, threshold1, threshold2)
```

- image = input image, normally grayscale.
 - threshold1 = lower hysteresis threshold.
 - threshold2 = upper hysteresis threshold.
 - returns an edge map.
-

6. Region Growing

Purpose: Start from a seed pixel and expand a region through sufficiently similar neighboring pixels.

```
import cv2
import numpy as np

# Read image as grayscale
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)

# Region growing function
def region_growing(image, seed, threshold):

    # Create empty segmentation mask
    mask = np.zeros_like(image, dtype=np.uint8)

    # Start with the seed point
    stack = [seed]

    # Use the seed intensity as the reference
    seed_intensity = image[seed]

    while stack:
        x, y = stack.pop()

        # Ignore coordinates outside the image
        if x < 0 or x >= image.shape[0] or y < 0 or y >= image.shape[1]:
            continue

        # Only process pixels not already included
        if mask[x, y] == 0:

            # Check similarity with the seed intensity
            if abs(int(image[x, y]) - int(seed_intensity)) < threshold:

                # Add this pixel to the region
                mask[x, y] = 255

                # Add 4-connected neighbors
                stack.extend([
                    (x + 1, y),
                    (x - 1, y),
                    (x, y + 1),
                    (x, y - 1)
                ])
```

```

    return mask

# Example seed point and threshold
seed_point = (10, 10)
threshold = 50

# Run region growing
segmented_image = region_growing(image, seed_point, threshold)

```

Function / argument dictionary

`np.zeros_like(a, dtype=...)`

- `a` = array whose shape is copied.
- `dtype` = output datatype; here `np.uint8`.

`region_growing(image, seed, threshold)`

- `image` = grayscale input.
- `seed` = starting (row, column) coordinate.
- `threshold` = maximum allowed absolute intensity difference from the seed in this implementation.

`image.shape[0]` = number of rows / height.

`image.shape[1]` = number of columns / width.

`stack.pop()` = removes and returns one stored coordinate.

`stack.extend(list)` = adds neighboring coordinates to the stack.

Exam recognition

Seed -> compare neighbors -> add similar pixels -> keep growing -> binary mask

7. Watershed Segmentation

Purpose: Separate touching/overlapping objects using markers and watershed boundaries.

```
import cv2
import numpy as np

# Read image
image = cv2.imread('image.jpg')

# Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Threshold using Otsu
_, thresh = cv2.threshold(
    gray, 0, 255,
    cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
)

# Remove small noise using morphological opening
kernel = np.ones((5, 5), np.uint8)
opening = cv2.morphologyEx(
    thresh,
    cv2.MORPH_OPEN,
    kernel,
    iterations=2
)

# Sure background
sure_bg = cv2.dilate(opening, kernel, iterations=3)

# Distance transform
dist_transform = cv2.distanceTransform(opening, cv2.DIST_L2, 5)

# Sure foreground
_, sure_fg = cv2.threshold(
    dist_transform,
    0.2 * dist_transform.max(),
    255,
    0
)

sure_fg = np.uint8(sure_fg)

# Unknown region
```

```

unknown = cv2.subtract(sure_bg, sure_fg)

# Label connected foreground regions
_, markers = cv2.connectedComponents(sure_fg)

# Shift labels and mark unknown area as zero
markers = markers + 1
markers[unknown == 255] = 0

# Apply watershed
cv2.watershed(image, markers)

# Mark watershed boundaries
image[markers == -1] = [255, 0, 0]

```

Function / argument dictionary

`cv2.cvtColor(src, code)`

- `code= cv2.COLOR_BGR2GRAY` -> BGR to grayscale.

`cv2.morphologyEx(src, op, kernel, iterations=...)`

- `src` = input binary image.
- `op` = morphology operation; manual uses `cv2.MORPH_OPEN`.
- `kernel` = structuring element.
- `iterations` = repeat count.

`np.ones(shape, dtype=...)`

- creates a kernel filled with ones.

`cv2.dilate(src, kernel, iterations=...)`

- expands bright regions; used here to estimate sure background.

`cv2.distanceTransform(src, distanceType, maskSize)`

- `src` = binary input.
- `distanceType` = e.g. `cv2.DIST_L2`.

- `maskSize` = distance mask size; manual uses `5` .

```
cv2.connectedComponents(image)
```

- labels connected regions.
- returns number of labels and label image.

```
cv2.subtract(src1, src2)
```

- subtracts two arrays pixelwise; used for the unknown region.

```
cv2.watershed(image, markers)
```

- `image` = original/color image.
- `markers` = integer marker map.
- modifies the marker image in place.
- boundary pixels are labeled `-1` .

Watershed memory chain

```
threshold  
-> opening / noise removal  
-> sure background  
-> distance transform  
-> sure foreground  
-> unknown  
-> connected components / markers  
-> watershed  
-> boundaries
```

8. Clustering-Based Segmentation - K-Means

Purpose: Group pixels into K clusters according to their feature/color similarity.

```
import cv2  
import numpy as np
```

```
# Read color image
image = cv2.imread('image.jpg')

# Turn image into a list of pixels, each with B,G,R values
pixel_values = image.reshape((-1, 3))

# Convert data to float32
pixel_values = np.float32(pixel_values)

# Define stopping criteria
criteria = (
    cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER,
    100,
    0.2
)

# Number of clusters
K = 4

# Run K-Means
_, labels, centers = cv2.kmeans(
    pixel_values,
    K,
    None,
    criteria,
    10,
    cv2.KMEANS_RANDOM_CENTERS
)

# Convert cluster centers back to 8-bit values
centers = np.uint8(centers)

# Replace each pixel by its cluster center
segmented_image = centers[labels.flatten()]

# Restore original image shape
segmented_image = segmented_image.reshape(image.shape)
```

Function / argument dictionary

```
image.reshape((-1, 3))
```

- `-1` = let NumPy infer the number of rows.
- `3` = three color features per pixel (B, G, R).

```
np.float32(a)
```

- converts array to 32-bit floating point.

```
cv2.kmeans(data, K, bestLabels, criteria, attempts, flags)
```

- `data` = samples/features.
- `K` = number of clusters.
- `bestLabels` = initial labels input; manual uses `None` .
- `criteria` = stopping criteria.
- `attempts` = number of independent initializations; manual uses `10` .
- `flags` = center initialization method; manual uses `cv2.KMEANS_RANDOM_CENTERS` .
- returns compactness, labels, centers.

```
labels.flatten() -> makes cluster labels 1D for indexing.
```

Exam changes

Change only:

```
K = 2  
K = 4  
K = 6
```

when asked to compare cluster counts.

PART B - LAB 5 TASKS

Task 1 - Document under uneven lighting

What the task wants

- Convert a document image to grayscale.

- Try at least 3 global thresholds.
- Compare their results.
- Apply adaptive thresholding.
- Decide why global thresholding struggles under uneven illumination.

Complete solution

```
import cv2
import matplotlib.pyplot as plt

# Load document as grayscale
image = cv2.imread('document.jpg', cv2.IMREAD_GRAYSCALE)

if image is None:
    raise SystemExit('Image not found')

# Try three global thresholds
_, global_1 = cv2.threshold(image, 80, 255, cv2.THRESH_BINARY)
_, global_2 = cv2.threshold(image, 120, 255, cv2.THRESH_BINARY)
_, global_3 = cv2.threshold(image, 160, 255, cv2.THRESH_BINARY)

# Use adaptive thresholding for uneven illumination
adaptive = cv2.adaptiveThreshold(
    image, 255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11, 2
)

# Display comparison
plt.figure(figsize=(12, 8))

plt.subplot(2, 3, 1)
plt.imshow(image, cmap='gray')
plt.title('Original')
plt.axis('off')

plt.subplot(2, 3, 2)
plt.imshow(global_1, cmap='gray')
plt.title('Global T=80')
plt.axis('off')
```

```
plt.subplot(2, 3, 3)
plt.imshow(global_2, cmap='gray')
plt.title('Global T=120')
plt.axis('off')

plt.subplot(2, 3, 4)
plt.imshow(global_3, cmap='gray')
plt.title('Global T=160')
plt.axis('off')

plt.subplot(2, 3, 5)
plt.imshow(adaptive, cmap='gray')
plt.title('Adaptive')
plt.axis('off')

plt.tight_layout()
plt.show()
```

Why this workflow?

grayscale -> global threshold experiments -> adaptive threshold -> visual comparison .

A single global threshold can fail when the same foreground object has different intensities because of different local lighting conditions.

Functions used

- `cv2.imread(filename, flags)`
- `cv2.threshold(src, thresh, maxval, type)`
- `cv2.adaptiveThreshold(src, maxValue, adaptiveMethod, thresholdType, blockSize, C)`
- `plt.figure(figsize=...)`
- `plt.subplot(...)`
- `plt.imshow(...)`
- `plt.title(...)`
- `plt.axis(...)`
- `plt.tight_layout()`

- `plt.show()`

Task 2 - Compare adaptive threshold strategies

What the task wants

- Test both Mean and Gaussian adaptive methods.
- Try at least 3 `blockSize` values.
- Try at least 3 `c` values.
- Compare the outputs and choose the cleanest setting.

Complete solution

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread('document.jpg', cv2.IMREAD_GRAYSCALE)
if image is None:
    raise SystemExit('Image not found')

block_sizes = [7, 11, 15] # Must be odd
c_values = [2, 5, 8]

results = []

# Mean-based experiments
for block in block_sizes:
    for c in c_values:
        result = cv2.adaptiveThreshold(
            image, 255,
            cv2.ADAPTIVE_THRESH_MEAN_C,
            cv2.THRESH_BINARY,
            block, c
        )
        results.append(('Mean', block, c, result))

# Gaussian-based experiments
for block in block_sizes:
    for c in c_values:
```

```

        result = cv2.adaptiveThreshold(
            image, 255,
            cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
            cv2.THRESH_BINARY,
            block, c
        )
        results.append(('Gaussian', block, c, result))

# Display a sample of the experiments
plt.figure(figsize=(14, 10))
plt.subplot(3, 3, 1)
plt.imshow(image, cmap='gray')
plt.title('Original')
plt.axis('off')

for i in range(8):
    method, block, c, result = results[i]
    plt.subplot(3, 3, i + 2)
    plt.imshow(result, cmap='gray')
    plt.title(f'{method}: B={block}, C={c}')
    plt.axis('off')

plt.tight_layout()
plt.show()

```

Main parameters

- `blockSize` -> local neighborhood size; must be odd and greater than 1.
- `C` -> offset in the adaptive threshold calculation.
- `ADAPTIVE_THRESH_MEAN_C` -> local mean-based method.
- `ADAPTIVE_THRESH_GAUSSIAN_C` -> Gaussian-weighted local method.

Functions used

`cv2.adaptiveThreshold(src, maxValue, adaptiveMethod, thresholdType, blockSize, C)` plus plotting functions.

Task 3 - Otsu + histogram

What the task wants

- Convert the image to grayscale.
- Generate the grayscale histogram.
- Run Otsu's thresholding without manually selecting the threshold.
- Change preprocessing/contrast and run Otsu again.
- Compare the selected thresholds.

Complete solution

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread('coins.jpg')
if image is None:
    raise SystemExit('Image not found')

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Histogram of grayscale intensities
plt.figure()
plt.hist(gray.ravel(), bins=256, range=[0, 256])
plt.title('Grayscale Histogram')
plt.xlabel('Intensity')
plt.ylabel('Number of Pixels')
plt.show()

# Otsu automatically chooses the threshold
threshold_1, otsu_1 = cv2.threshold(
    gray, 0, 255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

print('Original Otsu threshold:', threshold_1)

# Change contrast and apply Otsu again
contrast_image = cv2.convertScaleAbs(gray, alpha=1.3, beta=0)
```

```
threshold_2, otsu_2 = cv2.threshold(  
    contrast_image, 0, 255,  
    cv2.THRESH_BINARY + cv2.THRESH_OTSU  
)  
  
print('Contrast-adjusted Otsu threshold:', threshold_2)
```

Functions used

- `cv2.cvtColor(src, code)`
- `cv2.threshold(src, thresh, maxval, type)`
- `cv2.convertScaleAbs(src, alpha, beta)`
- `plt.hist(...)`

Task 4 - HSV color segmentation

What the task wants

- Convert the image to HSV.
- Start with a restrictive HSV range.
- Create a mask.
- Widen/tune the range.
- Create a final mask.
- Extract the selected object.

Complete solution

```
import cv2  
import numpy as np  
import matplotlib.pyplot as plt  
  
image = cv2.imread('colored_object.jpg')  
if image is None:
```

```
raise SystemExit('Image not found')

# Convert BGR to HSV
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

# Restrictive range
lower_restrictive = np.array([25, 150, 150])
upper_restrictive = np.array([30, 255, 255])
restrictive_mask = cv2.inRange(
    hsv,
    lower_restrictive,
    upper_restrictive
)

# Final wider range
lower_final = np.array([20, 80, 80])
upper_final = np.array([40, 255, 255])
final_mask = cv2.inRange(
    hsv,
    lower_final,
    upper_final
)

# Extract the object from the original image
selected_object = cv2.bitwise_and(
    image,
    image,
    mask=final_mask
)

# Display
plt.figure(figsize=(12, 8))
plt.subplot(2, 2, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title('Original')
plt.axis('off')

plt.subplot(2, 2, 2)
plt.imshow(restrictive_mask, cmap='gray')
plt.title('Restrictive Mask')
plt.axis('off')

plt.subplot(2, 2, 3)
plt.imshow(final_mask, cmap='gray')
plt.title('Final Mask')
plt.axis('off')
```

```
plt.subplot(2, 2, 4)
plt.imshow(cv2.cvtColor(selected_object, cv2.COLOR_BGR2RGB))
plt.title('Selected Object')
plt.axis('off')

plt.tight_layout()
plt.show()
```

Functions used

- `cv2.imread(filename, flags)`
- `cv2.cvtColor(src, code)`
- `np.array(object)`
- `cv2.inRange(src, lowerb, upperb)`
- `cv2.bitwise_and(src1, src2, mask=...)`
- plotting functions

Task 5 - Canny edge detection

What the task wants

- Convert/read image as grayscale.
- Try at least 3 low/high threshold pairs.
- Compare strong, weak, missing and unwanted edges.

Complete solution

```
import cv2
import matplotlib.pyplot as plt

image = cv2.imread('object.jpg', cv2.IMREAD_GRAYSCALE)
if image is None:
    raise SystemExit('Image not found')
```

```
edges_1 = cv2.Canny(image, 50, 150)
edges_2 = cv2.Canny(image, 100, 200)
edges_3 = cv2.Canny(image, 150, 250)

plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
plt.imshow(image, cmap='gray')
plt.title('Original')
plt.axis('off')

plt.subplot(2, 2, 2)
plt.imshow(edges_1, cmap='gray')
plt.title('Low=50 High=150')
plt.axis('off')

plt.subplot(2, 2, 3)
plt.imshow(edges_2, cmap='gray')
plt.title('Low=100 High=200')
plt.axis('off')

plt.subplot(2, 2, 4)
plt.imshow(edges_3, cmap='gray')
plt.title('Low=150 High=250')
plt.axis('off')

plt.tight_layout()
plt.show()
```

Functions used

- `cv2.Canny(image, threshold1, threshold2)`
- plotting functions

Parameter memory

`threshold1` = low, `threshold2` = high.

Task 6 - Region growing

What the task wants

- Work in grayscale.
- Choose a seed point.
- Grow a region using neighboring intensity similarity.
- Test multiple thresholds and seed locations.

Complete solution

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('medical_image.jpg', cv2.IMREAD_GRAYSCALE)
if image is None:
    raise SystemExit('Image not found')

def region_growing(image, seed, threshold):
    mask = np.zeros_like(image, dtype=np.uint8)
    stack = [seed]
    seed_intensity = image[seed]

    while stack:
        x, y = stack.pop()

        if x < 0 or x >= image.shape[0] or y < 0 or y >= image.shape[1]:
            continue

        if mask[x, y] == 0:
            if abs(int(image[x, y]) - int(seed_intensity)) < threshold:
                mask[x, y] = 255
                stack.extend([
                    (x + 1, y),
                    (x - 1, y),
                    (x, y + 1),
                    (x, y - 1)
                ])

    return mask
```

```
# Experiment with thresholds
seed_point = (10, 10)
thresholds = [20, 50, 80]

for threshold in thresholds:
    result = region_growing(image, seed_point, threshold)
    plt.figure()
    plt.imshow(result, cmap='gray')
    plt.title(f'Seed={seed_point}, Threshold={threshold}')
    plt.axis('off')
    plt.show()
```

Functions / operations used

- `np.zeros_like(a, dtype=...)`
- `image[seed]` -> read seed intensity.
- `stack.pop()` -> process a coordinate.
- `stack.extend(...)` -> add 4-connected neighbors.
- `abs(a-b)` -> intensity difference.

Main variables to change

- `seed_point`
- `threshold`

Task 7 - Watershed for touching objects

What the task wants

Use a marker-based watershed pipeline:

```
Preprocess -> threshold -> noise removal -> sure BG -> distance transform
-> sure FG -> unknown -> connected components/markers -> watershed
-> boundary visualization
```

Complete solution

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('coins.jpg')
if image is None:
    raise SystemExit('Image not found')

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Threshold using Otsu
_, thresh = cv2.threshold(
    gray, 0, 255,
    cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
)

# Noise removal
kernel = np.ones((5, 5), np.uint8)
opening = cv2.morphologyEx(
    thresh,
    cv2.MORPH_OPEN,
    kernel,
    iterations=2
)

# Sure background
sure_bg = cv2.dilate(
    opening,
    kernel,
    iterations=3
)

# Distance transform
dist_transform = cv2.distanceTransform(
    opening,
    cv2.DIST_L2,
    5
)

# Sure foreground
_, sure_fg = cv2.threshold(
    dist_transform,
    0.2 * dist_transform.max(),
    255,
```

```
    0
)
sure_fg = np.uint8(sure_fg)

# Unknown region
unknown = cv2.subtract(sure_bg, sure_fg)

# Marker labeling
_, markers = cv2.connectedComponents(sure_fg)
markers = markers + 1
markers[unknown == 255] = 0

# Watershed
result = image.copy()
cv2.watershed(result, markers)

# Highlight boundaries
result[markers == -1] = [255, 0, 0]

plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.title('Watershed Result')
plt.axis('off')
plt.show()
```

Functions used

- `cv2.cvtColor(src, code)`
- `cv2.threshold(src, thresh, maxval, type)`
- `np.ones(shape, dtype)`
- `cv2.morphologyEx(src, op, kernel, iterations)`
- `cv2.dilate(src, kernel, iterations)`
- `cv2.distanceTransform(src, distanceType, maskSize)`
- `cv2.subtract(src1, src2)`
- `cv2.connectedComponents(image)`
- `cv2.watershed(image, markers)`

Crucial markers fact

After watershed, boundary pixels in `markers` are labeled `-1`.

Task 8 - Tune watershed distance threshold

What the task wants

Keep the same watershed pipeline but change the distance-transform threshold. Record how marker count and separated regions change.

Complete solution

```
import cv2
import numpy as np

image = cv2.imread('coins.jpg')
if image is None:
    raise SystemExit('Image not found')

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

_, thresh = cv2.threshold(
    gray, 0, 255,
    cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
)

kernel = np.ones((5, 5), np.uint8)
opening = cv2.morphologyEx(
    thresh, cv2.MORPH_OPEN, kernel, iterations=2
)

sure_bg = cv2.dilate(opening, kernel, iterations=3)

dist_transform = cv2.distanceTransform(
    opening, cv2.DIST_L2, 5
)

for ratio in [0.10, 0.20, 0.30, 0.40]:
    # Change the sure-foreground threshold
    _, sure_fg = cv2.threshold(
        dist_transform,
```

```

        ratio * dist_transform.max(),
        255,
        0
    )
    sure_fg = np.uint8(sure_fg)

    unknown = cv2.subtract(sure_bg, sure_fg)

    _, markers = cv2.connectedComponents(sure_fg)
    foreground_markers = markers.max()

    markers = markers + 1
    markers[unknown == 255] = 0

    result = image.copy()
    cv2.watershed(result, markers)

    labels = np.unique(markers[markers > 1])
    separated_regions = len(labels)

    print('Distance threshold:', ratio)
    print('Foreground markers:', foreground_markers)
    print('Separated regions:', separated_regions)
    print('-----')

```

Main parameter

```
ratio * dist_transform.max()
```

Increasing/decreasing this changes the sure-foreground area, which changes the marker configuration and therefore the watershed result.

Task 9 - K-Means with K=2,4,6

What the task wants

- Convert the image to a pixel list.
- Convert pixels to float32.
- Run K-Means with multiple K values.

- Reconstruct the segmented images.
- Compare the level of detail.

Complete solution

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('scene.jpg')
if image is None:
    raise SystemExit('Image not found')

# Reshape image: one row per pixel, 3 color features
pixels = image.reshape((-1, 3))
pixels = np.float32(pixels)

criteria = (
    cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER,
    100,
    0.2
)

for K in [2, 4, 6]:
    _, labels, centers = cv2.kmeans(
        pixels,
        K,
        None,
        criteria,
        10,
        cv2.KMEANS_RANDOM_CENTERS
    )

    centers = np.uint8(centers)
    segmented = centers[labels.flatten()]
    segmented = segmented.reshape(image.shape)

    plt.figure()
    plt.imshow(cv2.cvtColor(segmented, cv2.COLOR_BGR2RGB))
    plt.title(f'K = {K}')
    plt.axis('off')
    plt.show()
```

Functions used

- `reshape((-1,3))`
- `np.float32(...)`
- `cv2.kmeans(data, K, bestLabels, criteria, attempts, flags)`
- `np.uint8(...)`
- `labels.flatten()`
- `reshape(image.shape)`

K memory

- smaller K -> fewer clusters / stronger color simplification.
- larger K -> more clusters / more retained detail.

Task 10 - Compare three segmentation methods

What the task wants

For an image of your choice, compare 3 methods and explain:

1. What assumption each method makes.
2. What it identifies successfully.
3. What it segments incorrectly.
4. Which parameter has the biggest effect.
5. Which method is best based on the observed result.

Example complete solution: Global + HSV + K-Means

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```

image = cv2.imread('colored_object.jpg')
if image is None:
    raise SystemExit('Image not found')

# -----
# Method 1: Global threshold
# -----
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
_, global_mask = cv2.threshold(
    gray, 128, 255, cv2.THRESH_BINARY
)

# -----
# Method 2: HSV color threshold
# -----
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
lower = np.array([20, 80, 80])
upper = np.array([40, 255, 255])
color_mask = cv2.inRange(hsv, lower, upper)

# -----
# Method 3: K-Means
# -----
pixels = image.reshape((-1, 3))
pixels = np.float32(pixels)

criteria = (
    cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER,
    100,
    0.2
)

K = 4
_, labels, centers = cv2.kmeans(
    pixels, K, None, criteria, 10,
    cv2.KMEANS_RANDOM_CENTERS
)

centers = np.uint8(centers)
kmeans_result = centers[labels.flatten()]
kmeans_result = kmeans_result.reshape(image.shape)

# Display comparison
plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)

```

```
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title('Original')
plt.axis('off')

plt.subplot(2, 2, 2)
plt.imshow(global_mask, cmap='gray')
plt.title('Global Threshold')
plt.axis('off')

plt.subplot(2, 2, 3)
plt.imshow(color_mask, cmap='gray')
plt.title('HSV Color Threshold')
plt.axis('off')

plt.subplot(2, 2, 4)
plt.imshow(cv2.cvtColor(kmeans_result, cv2.COLOR_BGR2RGB))
plt.title('K-Means')
plt.axis('off')

plt.tight_layout()
plt.show()
```

How to write the analysis

Global Threshold: assumes object/background differ in grayscale intensity. Main parameter = threshold.

HSV Color Threshold: assumes desired object has a distinguishable color range. Main parameters = lower/upper HSV bounds.

K-Means: assumes pixels can be grouped meaningfully by feature/color similarity. Main parameter = K.

Pick the best method from the actual output, not from the method name.

PART C - MASTER FUNCTION REFERENCE

Image input

```
cv2.imread(filename, flags)
```

- `filename` = path/name.
- `flags` = read mode; `cv2.IMREAD_GRAYSCALE` for grayscale.

Global threshold

```
cv2.threshold(src, thresh, maxval, type)
```

- `src` = input image.
- `thresh` = manual/global threshold or `0` when using Otsu.
- `maxval` = output maximum value.
- `type` = `cv2.THRESH_BINARY`, `cv2.THRESH_BINARY_INV`, etc.

Adaptive threshold

```
cv2.adaptiveThreshold(src, maxValue, adaptiveMethod, thresholdType, blockSize, C)
```

- `adaptiveMethod` = `MEAN_C` or `GAUSSIAN_C`.
- `blockSize` = local neighborhood size; odd.
- `C` = local threshold offset.

Color conversion

```
cv2.cvtColor(src, code)
```

- `code= cv2.COLOR_BGR2HSV` -> BGR to HSV.
- `code= cv2.COLOR_BGR2GRAY` -> BGR to grayscale.

Color range mask

```
cv2.inRange(src, lowerb, upperb)
```

- `lowerb` = lower channel bounds.
- `upperb` = upper channel bounds.
- output = binary mask.

Canny

```
cv2.Canny(image, threshold1, threshold2)
```

- first = low.
- second = high.
- output = edge map.

Morphology

```
cv2.morphologyEx(src, op, kernel, iterations=...)
```

- `op= cv2.MORPH_OPEN` -> opening.
- opening = erosion followed by dilation.

Dilation

```
cv2.dilate(src, kernel, iterations=...)
```

- expands bright/foreground regions.

Distance transform

```
cv2.distanceTransform(src, distanceType, maskSize)
```

- `distanceType= cv2.DIST_L2` in the manual.
- `maskSize=5` in the manual code.

Connected components

```
cv2.connectedComponents(image)
```

- labels each connected foreground component.

Watershed

```
cv2.watershed(image, markers)
```

- uses marker map to segment.
- boundary pixels become `-1` .

K-Means

```
cv2.kmeans(data, K, bestLabels, criteria, attempts, flags)
```

- `data` = feature samples.
- `K` = number of clusters.
- `bestLabels` = initial labels input or `None` .
- `criteria` = stopping rule.
- `attempts` = number of initializations.
- `flags` = initialization strategy.

NumPy helpers

```
np.array(values)
np.ones(shape, dtype=...)
np.zeros_like(array, dtype=...)
np.float32(array)
np.uint8(array)
np.ravel(array)
np.unique(array)
```

Plotting

```
plt.figure(figsize=(w, h))
plt.subplot(rows, cols, index)
plt.imshow(image, cmap='gray')
plt.title('...')
plt.axis('off')
plt.tight_layout()
plt.show()
```

PART D - IF SIR CHANGES THE WORDING

If the question says...	Start with...
simple binary separation by intensity	<code>cv2.threshold()</code>
uneven illumination / shadows	<code>cv2.adaptiveThreshold()</code>
automatic threshold selection	<code>cv2.THRESH_OTSU</code>
segment a specific color	<code>BGR -> HSV -> cv2.inRange()</code>
find object boundaries/edges	<code>cv2.Canny()</code>
grow a region from a starting pixel	<code>region_growing()</code>
separate touching objects	<code>cv2.watershed()</code> pipeline
group similar colors/features	<code>cv2.kmeans()</code>

PART E - 30-SECOND CODING CHECKLIST

1. What type of image does my method need?
2. What function does the actual segmentation?
3. What are the arguments in the correct order?
4. Which parameter is sir likely to change?
5. What output does the function return?
6. Do I need a mask, edge map, markers, or reconstructed image?
7. Does the final image need to be displayed/saved?

PART F - FAST MCQ MEMORY

Global threshold	-> one threshold for whole image
Adaptive threshold	-> local threshold per neighborhood
Otsu	-> automatically chooses global threshold
blockSize	-> adaptive neighborhood size
C	-> adaptive threshold offset
HSV	-> useful for color-range segmentation
inRange	-> creates range-based mask
Canny	-> low + high thresholds
Region growing	-> seed + neighbors + similarity
Watershed	-> markers + touching-object separation
Distance transform	-> helps find sure foreground / object centers
connectedComponents	-> labels connected regions
watershed boundaries	-> marker value -1
K-Means	-> K clusters
labels	-> cluster assignment per sample/pixel
centers	-> cluster representatives
reshape((-1,3))	-> one BGR pixel per row
float32	-> datatype used before K-Means